MyDiscord Project Structure

Overview

This document outlines the project structure for the MyDiscord application, a Discord-like chat application implemented in C with PostgreSQL and GTK.

Directory Structure

```
mydiscord/
├── src/                    # Source code
│   ├── main.c              # Entry point for the application
│   ├── server/             # Server-side code
│   │   ├── server.c        # Server initialization and management
│   │   ├── server.h
│   │   ├── socket_handler.c  # Socket communication handling
│   │   ├── socket_handler.h
│   │   ├── thread_pool.c     # Thread management for client connections
│   │   └── thread_pool.h
│   ├── client/             # Client-side code
│   │   ├── client.c        # Client initialization and management
│   │   ├── client.h
│   │   ├── socket_client.c   # Client socket handling
│   │   └── socket_client.h
│   ├── db/                 # Database interactions
│   │   ├── db_conn.c       # Database connection management
│   │   ├── db_conn.h
│   │   ├── user_dao.c      # Data access objects for users
│   │   ├── user_dao.h
│   │   ├── channel_dao.c     # Data access objects for channels
│   │   ├── channel_dao.h
│   │   ├── message_dao.c     # Data access objects for messages
│   │   ├── message_dao.h
│   │   ├── reaction_dao.c    # Data access objects for reactions
│   │   └── reaction_dao.h
│   ├── models/             # Data structures
│   │   ├── user.h          # User structure and functions
│   │   ├── channel.h       # Channel structure and functions
│   │   ├── message.h       # Message structure and functions
│   │   ├── reaction.h      # Reaction structure and functions
│   │   └── role.h          # Role and permissions structures
│   ├── ui/                 # GTK UI components
│   │   ├── ui_manager.c      # Main UI management
│   │   ├── ui_manager.h
```

```
│   │   ├── login_window.c    # Login and registration UI
│   │   ├── login_window.h
│   │   ├── main_window.c     # Main application window
│   │   ├── main_window.h
│   │   ├── channel_list.c    # Channel listing component
│   │   └── channel_list.h
│   ├── utils/              # Utility functions
│   │   ├── config.c          # Configuration management
│   │   ├── config.h
│   │   ├── logger.c          # Logging functionality
│   │   ├── logger.h
│   │   ├── encryption.c      # Encryption/decryption utilities
│   │   ├── encryption.h
│   │   ├── auth.c            # Authentication utilities
│   │   └── auth.h
│   └── protocol/             # Communication protocol
│       ├── message_format.c  # Message formatting for network transmission
│       ├── message_format.h
│       ├── packet.c          # Network packet handling
│       └── packet.h
├── include/                # Public API headers
├── lib/                    # Libraries
├── bin/                    # Compiled binaries
├── build/                  # Build files
├── tests/                  # Test code
│   ├── unit/               # Unit tests
│   └── integration/        # Integration tests
├── docs/                   # Documentation
├── scripts/                # Build and utility scripts
├── db_scripts/             # Database creation scripts
│   └── myDiscord.sql         # Initial database setup
├── Makefile                # Build configuration
└── README.md               # Project documentation
```

Component Explanation

Main Source Directories

1. **src/**: Contains all source code for the application
    - **server/**: Handles network communications on the server side
    - **client/**: Manages client connections to the server
    - **db/**: Contains all database interactions using the DAO pattern
    - **models/**: Defines core data structures that represent the application's domain
    - **ui/**: Implements the GTK user interface components
    - **utils/**: Houses utility functions like logging and encryption

    - **protocol/**: Defines the communication protocol between client and server
2. **include/**: Contains public API headers for external use
3. **lib/**: External libraries used by the application
4. **bin/**: Directory for compiled binaries
5. **build/**: Contains build artifacts
6. **tests/**: Contains test code
    - **unit/**: Unit tests for individual components
    - **integration/**: Integration tests for component interaction
7. **docs/**: Project documentation
8. **scripts/**: Build and utility scripts
9. **db_scripts/**: Database creation and migration scripts

Key Design Elements

1. **Modularity**: Each component has a single responsibility for easier maintenance.
2. **Separation of Concerns**:
    - UI logic is separated from business logic
    - Network communication is isolated from data persistence
    - Data models are independent of their storage mechanism
3. **Data Access Abstraction**: DAO pattern abstracts database operations
4. **Security**: Authentication and encryption are built into the system architecture
5. **Concurrency**: Thread pool and socket handling for multiple simultaneous users
6. **Protocol Design**: Clear communication protocol between client and server